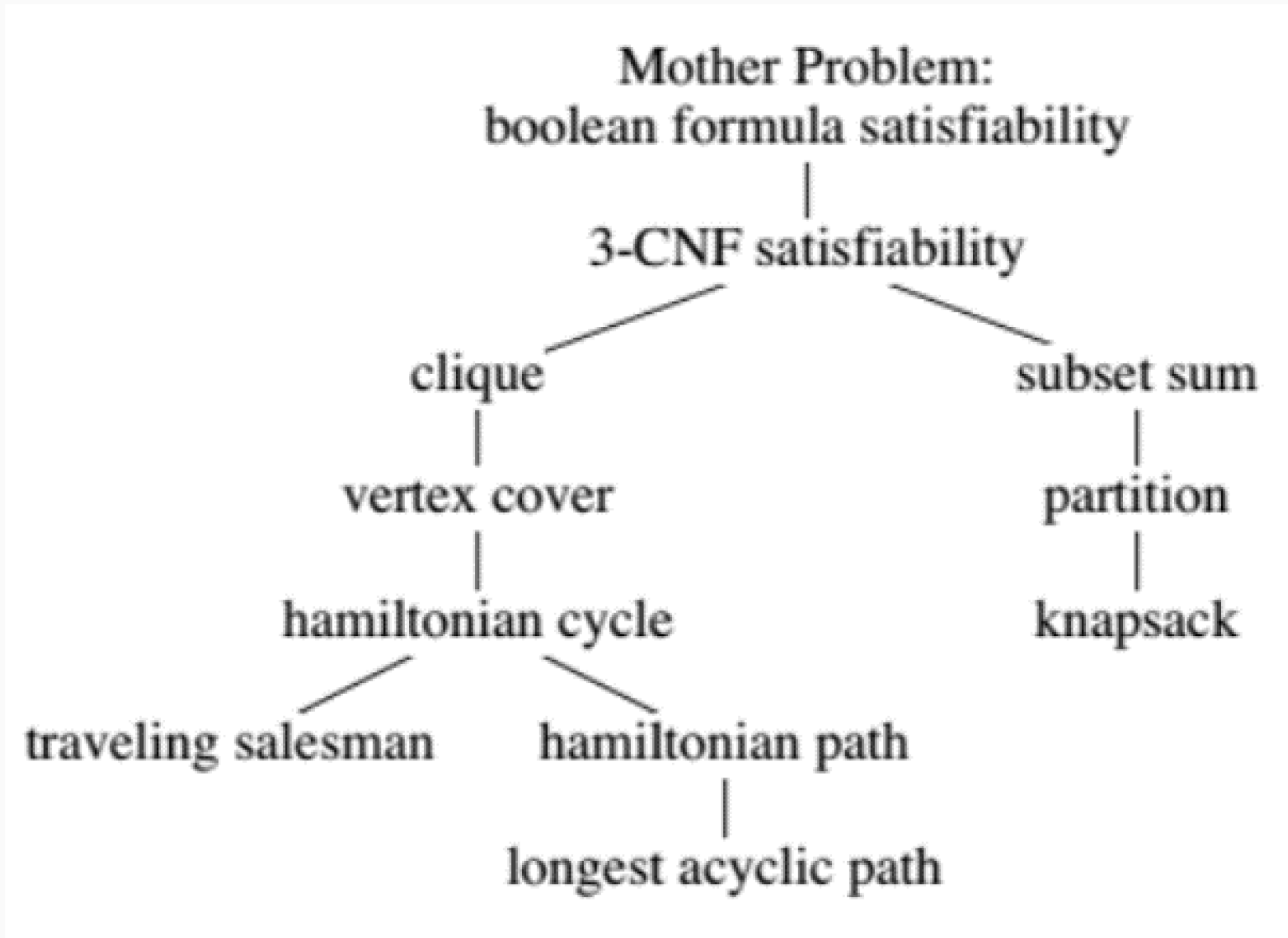


NP Complete Problems

Module 7

- SAT is the first problem that was proven to be NPC
- It is the mother problem for all NPC problems



(2) 3-Conjunctive Normal Form Satisfiability (3-CNF-SAT)

- Reduction proofs require that we handle any possible case of a known NPC problem
- A **literal** in a boolean formula is an occurrence of a variable or its negation.
- A boolean formula is in **conjunctive normal form (CNF)** if it is a conjunction of **clauses**, each of which is the disjunction of one or more literals.
- A boolean formula is in **3-conjunctive normal form (3-CNF)** if each clause has exactly three distinct literals. For example:
- $(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4)$

Continue...

- **3-CNF-SAT** asks whether a **boolean formula is satisfiable by an assignment of truth values to the variables**.
- 1) There are an exponential possible number of variable assignments.
- 2) But a given one instance can be **checked** in polynomial time merely by substituting and evaluating the expression.
- 3) SAT can be reduced to 3-CNF-SAT through a polynomial-time process

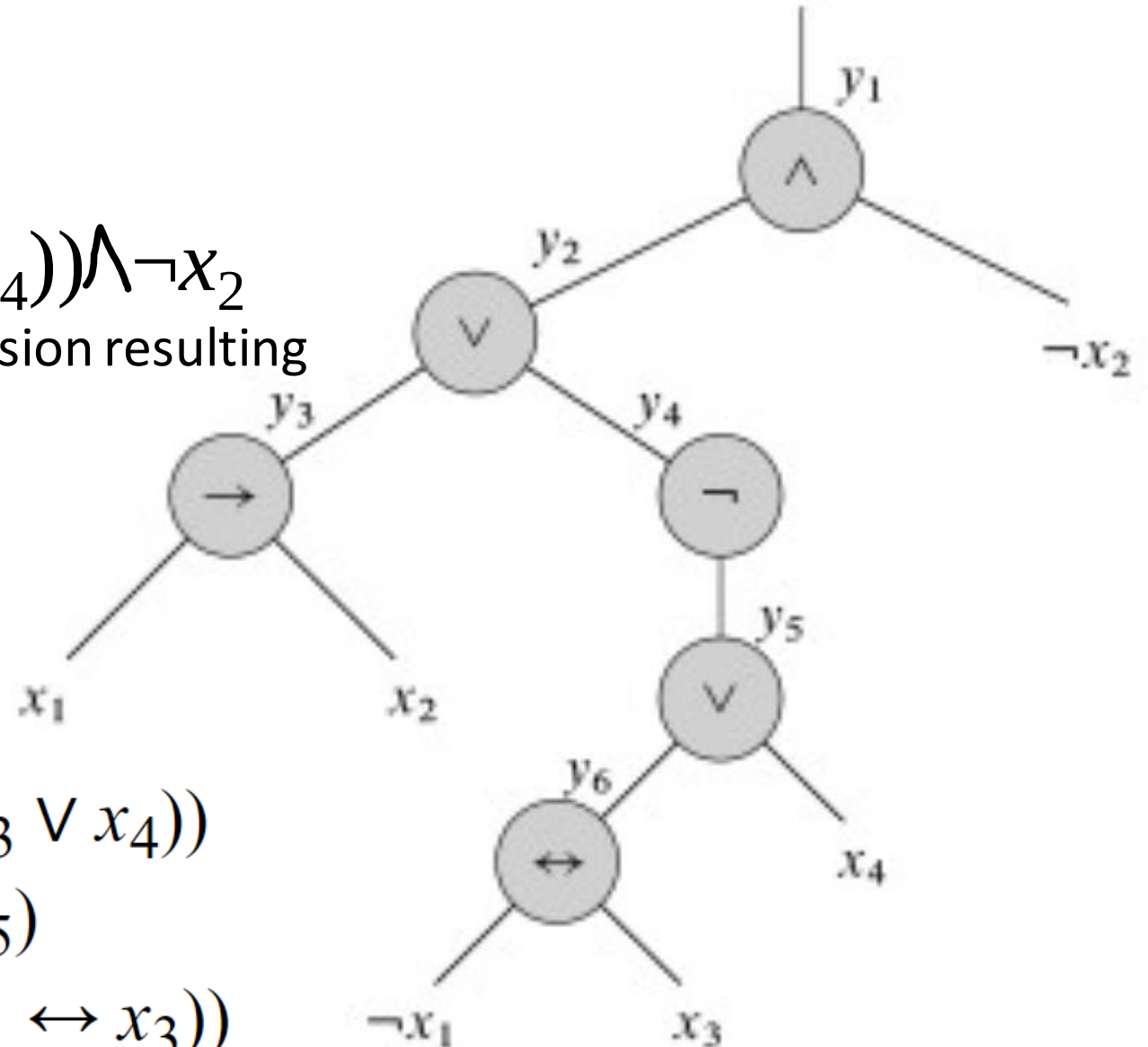
SAT to 3SAT Reduction

SAT can be reduced to 3-CNF-SAT through a polynomial-time process of:

- Parsing the SAT expression into a binary tree with literals as leaves and connectives as internal nodes;
- Introducing a variable y_i for the output of each internal node;
- Rewriting as the conjunction of the root variable and a clause for each node of the binary tree ($y_i \leftrightarrow$ the literal for its child nodes);
- Converting each clause to conjunctive normal form, first by converting to disjunctive normal form and then applying DeMorgan's laws to convert to CNF;
- Supply dummy variables as needed to convert clauses of 1 or 2 variables into 3-CNF.

SAT to 3SAT Reduction

$((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$
 the tree is shown to the right and the expression resulting
 from the tree is shown below.



$$\begin{aligned}
 &y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \wedge (y_2 \leftrightarrow (y_3 \vee x_4)) \\
 &\wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \wedge (y_4 \leftrightarrow \neg y_5) \\
 &\wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))
 \end{aligned}$$

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

$$\text{DNF} = (y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2)$$

Negating and applying DeMorgan's laws, we get the CNF formula

$$\begin{aligned} \phi_1'' = & (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2), \end{aligned}$$

Reduction to 3 CNF

At this point, we have converted each clause ϕ'_i of the formula ϕ' into a CNF formula ϕ''_i , and thus ϕ' is equivalent to the CNF formula ϕ'' consisting of the conjunction of the ϕ''_i . Moreover, each clause of ϕ'' has at most 3 literals.

- If C_i has 3 distinct literals, then simply include C_i as a clause of ϕ''' .
- If C_i has 2 distinct literals, that is, if $C_i = (l_1 \vee l_2)$, where l_1 and l_2 are literals, then include $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$ as clauses of ϕ''' . The literals p and $\neg p$ merely fulfill the syntactic requirement that each clause of ϕ''' has exactly 3 distinct literals. Whether $p = 0$ or $p = 1$, one of the clauses is equivalent to $l_1 \vee l_2$, and the other evaluates to 1, which is the identity for AND.
- If C_i has just 1 distinct literal l , then include $(l \vee p \vee q) \wedge (l \vee p \vee \neg q) \wedge (l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q)$ as clauses of ϕ''' . Regardless of the values of p and q , one of the four clauses is equivalent to l , and the other 3 evaluate to 1.

The remainder of the conversion uses De Morgan's laws :

$$\neg(a \wedge b) \equiv \neg a \vee \neg b$$

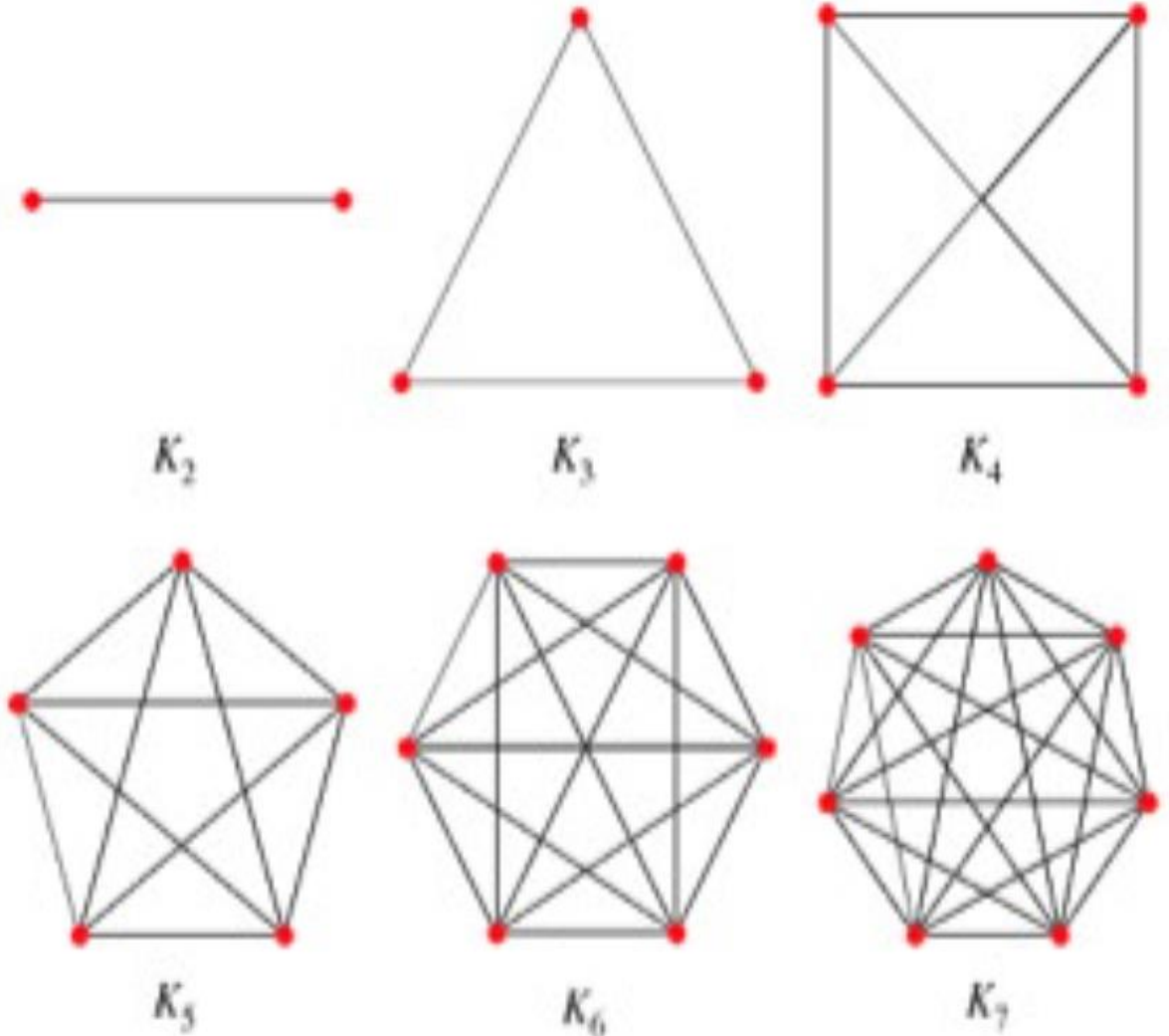
$$\neg(a \vee b) \equiv \neg a \wedge \neg b$$

resulting in:

$$(\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2)$$

CLIQUE

- A **clique** in an undirected graph $G = (V, E)$ is a subset $V' \subseteq V$, each pair of which is connected by an edge in E (a complete subgraph of G)
- The **clique problem** is the problem of finding a clique of maximum size in G . This can be converted to a decision problem by asking whether a clique of a given size k exists in the graph:
- $\text{CLIQUE} = \{ \langle G, k \rangle : G \text{ is a graph containing a clique of size } k \}$



(2) CLIQUE is in NP complete?

1. Is in NP?

- To find cliques in a graph with n vertices in time $O(3^{n/3})$, which is very much not a polynomial.
- It can be shown in $O(n^2)$ time that the given subgraph is a clique/not

2. There is an NP complete problem, 3-SAT that reduces to CLIQUE in polynomial time

Given an arbitrary formula ϕ in 3-conjunctive normal form with k clauses, we construct a graph G and ask if it has a clique of size k as follows:

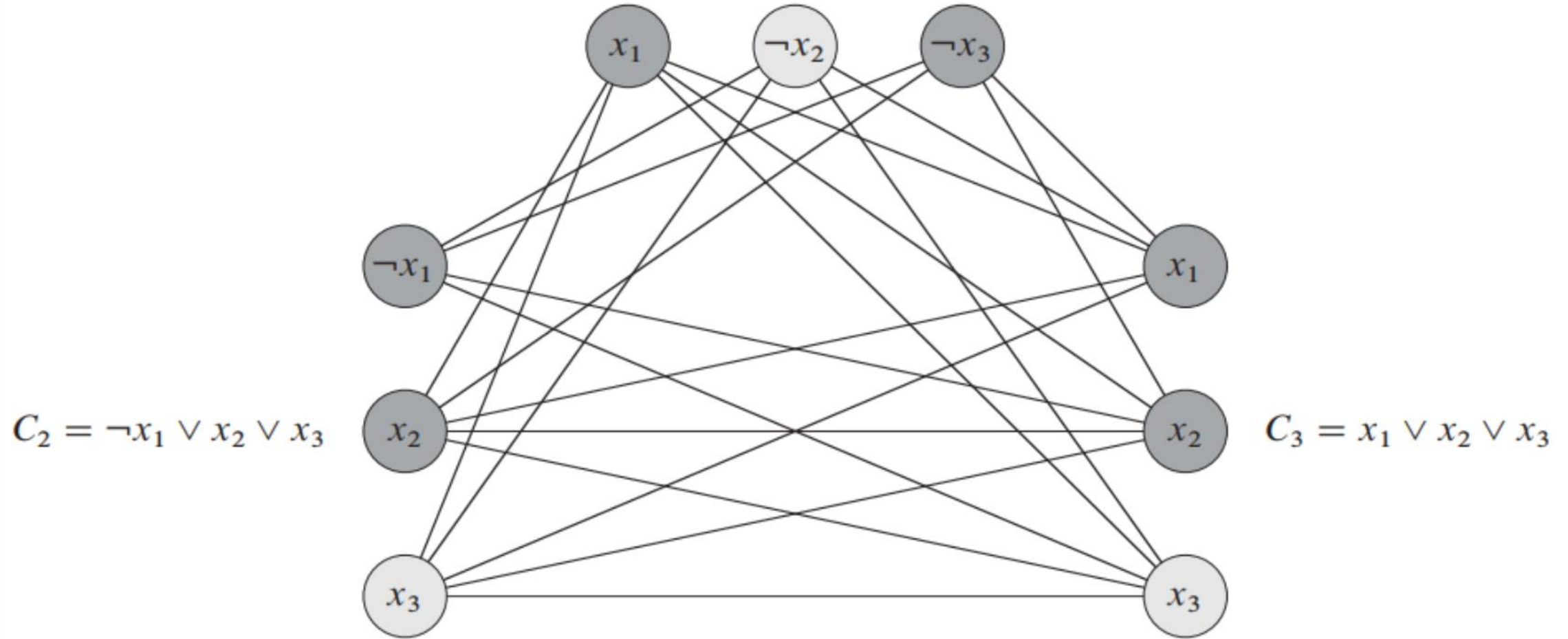
- We create a vertex for every literal in ϕ
- For every pair of vertices, we create an edge between them if the corresponding literals are in different triples *and* the literals are consistent (i.e., one is not the negation of the other).

- If there are k clauses in ϕ , we ask whether the graph has a k -clique. The claim is that such a clique exists in G **if and only if** there is a satisfying assignment for ϕ :
- **If direction** (*If clique exists in G , then there is a satisfying assignment in φ*):
 - By definition of k -clique, existence of a k -clique implies there are k vertices in G that are all connected to each other.
 - By our construction, the fact that two vertices are connected to each other means that they can receive a consistent boolean assignment (we can assign 1 to all of them), *and* that they are in different clauses.
 - Since there are k vertices in the clique, then at least one literal in each of the k clauses can be assigned a 1, i.e., the formula ϕ can be satisfied.

- **Only if direction** (*If ϕ can be satisfied, then there is a clique in G*):
 - If ϕ can be satisfied, then we can assign values to the literals, such that at least one literal in each clause is assigned value 1. i.e. they are consistent.
 - Consider the vertices corresponding to those literals. Since the literals are consistent and they are in different clauses, there is **an edge between every pair of them.**
 - Since there are **k clauses in ϕ** we have a **subset of at least k vertices in the graph** with edges between every pair of vertices, i.e., we have a k -clique in G .

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

$$C_1 = x_1 \vee \neg x_2 \vee \neg x_3$$



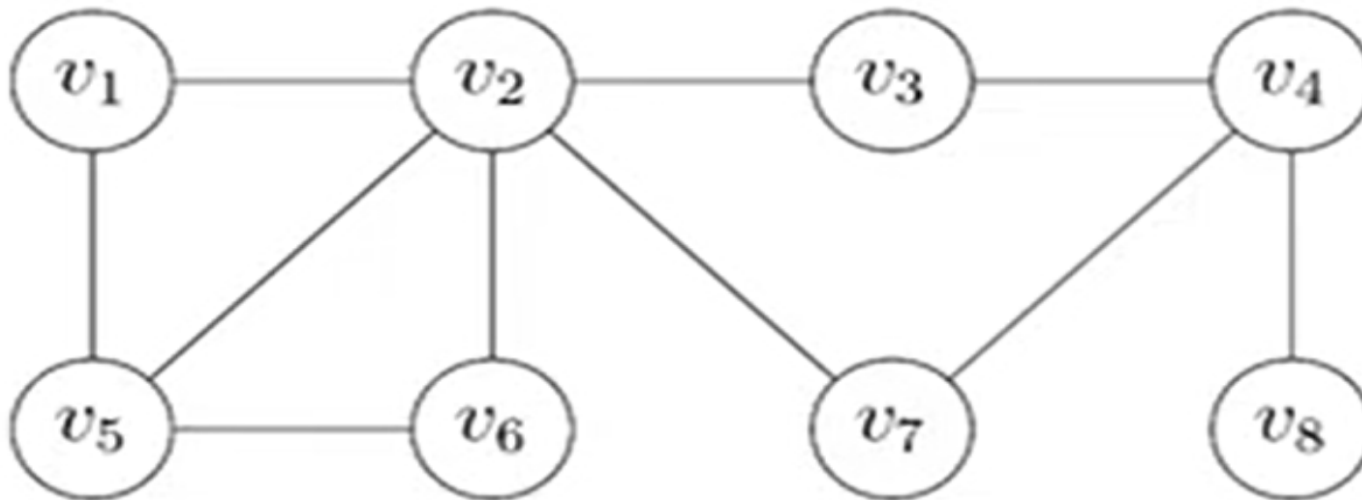
- Consider a satisfying assignment for ϕ , $x_2 = 0$ and $x_3 = 1$, $x_1 = 1$ or 0
- Pick a **True** literal from each clause, corresponding to this assignment and build V'
- V' will be a clique
- Consider the vertex $\neg x_2$ from C_1 , x_3 from C_2 and x_3 from C_3 , it is a **clique** (lightly shaded vertices)

- Conversely, take any clique from the graph,
 $V' = \{x_1, x_2, x_3\}$
- $x_1 = 1, x_2 = 1, x_3 = 1$ will be a satisfying assignment for
 $\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$
- $\therefore$ Satisfying assignment in $\phi \Leftrightarrow$ Clique in G
- $\therefore$ An instance of 3-CNF-SAT is reduced to CLIQUE in polynomial time
- $\therefore$ CLIQUE is NPC

- Any arbitrary instance of 3-CNF-SAT can be converted to an instance of CLIQUE in polynomial time with this particular structure.
- That means if we can solve **CLIQUE** we can solve any instance of 3-CNF-SAT, and since we know that 3-CNF-SAT is NPC, transitively, we can solve any instance in NP in polynomially related time.

Independent Set

- An independent set is a subset V' of the vertices of graph $G(V, E)$ such that if $u, v \in V'$, then $(u, v) \notin E$
- $\{v_1\}$, $\{v_2, v_4\}$, $\{v_1, v_6, v_3, v_8\}$ are independent sets
- $\{v_1, v_6, v_4, v_7\}$ is not an independent set



Independent Set problem

- Optimization $\longrightarrow$ Determine the largest independent set
- Decision $\longrightarrow$ Given a graph G and an integer K , does G contain an independent set of size K ?
- The decision version is known as the Independent Set Problem (ISP)
- The ISP is NPC

(3) ISP is NP Complete?

- ISP is NP?
 - To list out all combination of vertices and check whether they form an independent set of size K will take exponential time
 - A solution to ISP is a subset V' of the vertices of $G(V, E)$. Let $n = |V|$, $m = |E|$
 - Given V' , we first verify it contains K elements
 - For each pair $u, v \in V'$, check that $(u, v) \in E$.
 - There are $O(n^2)$ pair of vertices in V' , so this takes $O(mn^2)$ which is polynomial in m and n

3SAT $\leq_P$ Independent Set

The reduction 3SAT $\leq_P$ Independent Set

Input: Given a 3CNF formula φ

Goal: Construct a graph G_φ and number k such that G_φ has an independent set of size k if and only if φ is satisfiable.

G_φ should be constructable in time polynomial in size of φ

Importance of reduction: Although 3SAT is much more expressive, it can be reduced to a seemingly specialized Independent Set problem.

Notice: We handle only 3CNF formulas – reduction would not work for other kinds of boolean formulas.

Interpreting 3SAT

There are two ways to think about 3SAT

1. Find a way to assign 0/1 (false/true) to the variables such that the formula evaluates to true, that is each clause evaluates to true.
2. Pick a literal from each clause and find a truth assignment to make all of them true. You will fail if two of the literals you pick are in conflict, i.e., you pick x_i and $\neg x_i$

We will take the second view of 3SAT to construct the reduction.

The Reduction

1. G_φ will have one vertex for each literal in a clause
2. Connect the 3 literals in a clause to form a triangle; the independent set will pick at most one vertex from each clause, which will correspond to the literal to be set to true
3. Connect 2 vertices if they label complementary literals; this ensures that the literals corresponding to the independent set do not have a conflict
4. Take k to be the number of clauses

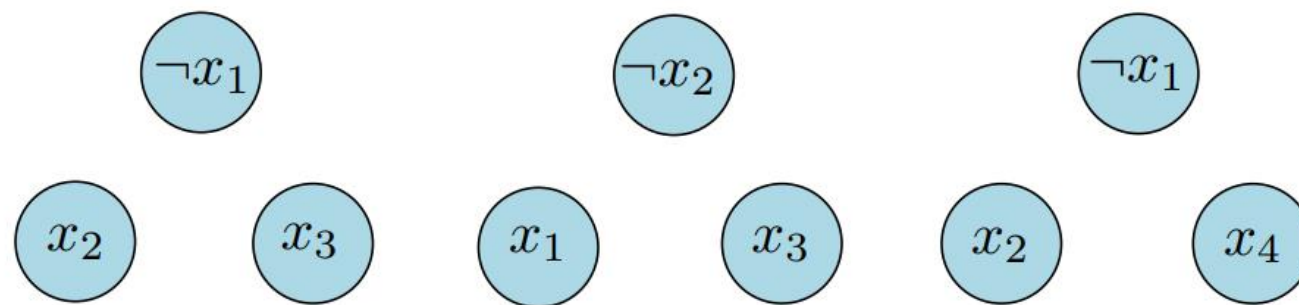


Figure: Graph for $\varphi = (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$

The Reduction

1. $\mathbf{G}_\varphi$ will have one vertex for each literal in a clause
2. Connect the 3 literals in a clause to form a triangle; the independent set will pick at most one vertex from each clause, which will correspond to the literal to be set to true
3. Connect 2 vertices if they label complementary literals; this ensures that the literals corresponding to the independent set do not have a conflict
4. Take k to be the number of clauses

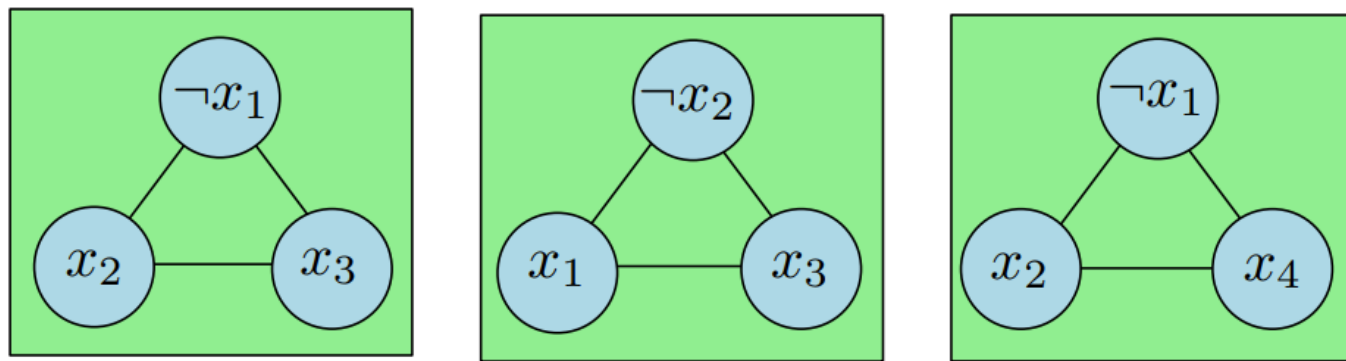


Figure: Graph for $\varphi = (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$

The Reduction

1. G_φ will have one vertex for each literal in a clause
2. Connect the 3 literals in a clause to form a triangle; the independent set will pick at most one vertex from each clause, which will correspond to the literal to be set to true
3. Connect 2 vertices if they label complementary literals; this ensures that the literals corresponding to the independent set do not have a conflict
4. Take k to be the number of clauses

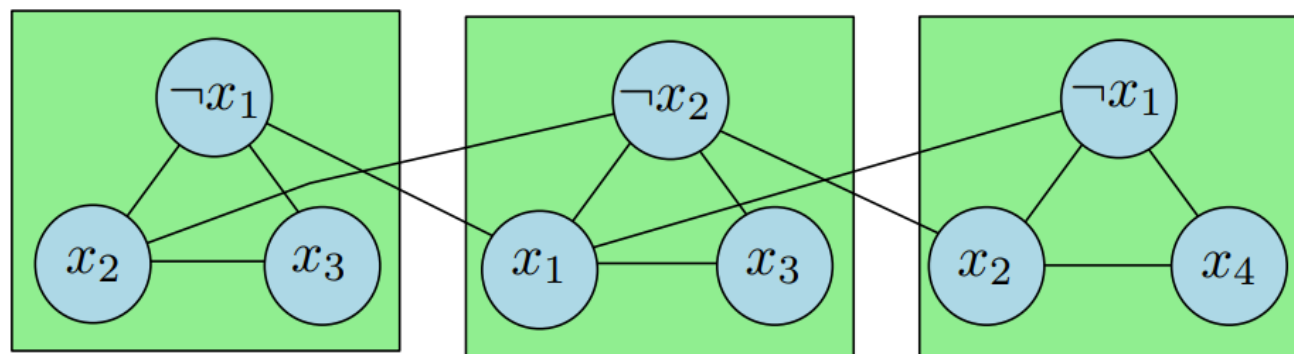


Figure: Graph for $\varphi = (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$

The Reduction

1. G_φ will have one vertex for each literal in a clause
2. Connect the 3 literals in a clause to form a triangle; the independent set will pick at most one vertex from each clause, which will correspond to the literal to be set to true
3. Connect 2 vertices if they label complementary literals; this ensures that the literals corresponding to the independent set do not have a conflict
4. Take k to be the number of clauses

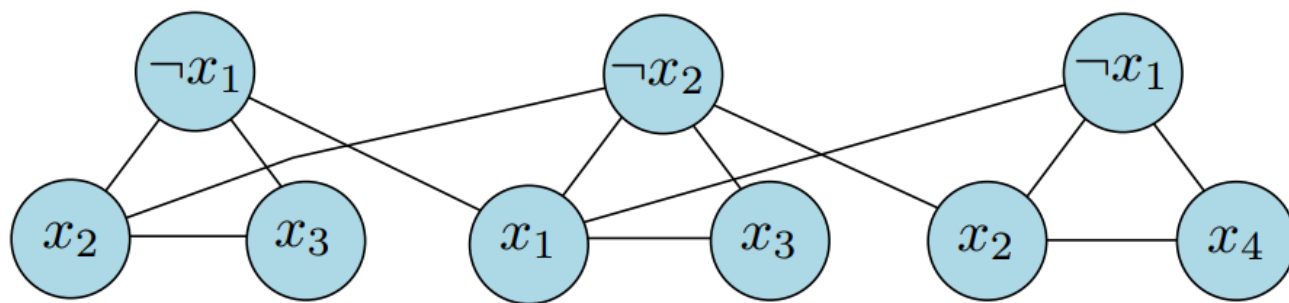


Figure: Graph for $\varphi = (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$

φ is satisfiable iff $\mathbf{G}_\varphi$ has an independent set of size k (= number of clauses in φ).

Proof.

$\Rightarrow$ Let $\mathbf{a}$ be the truth assignment satisfying φ

- Pick one of the vertices, corresponding to true literals under $\mathbf{a}$, from each triangle.
This is an independent set of the appropriate size. Why?

φ is satisfiable iff G_φ has an independent set of size k (= number of clauses in φ).

Proof.

$\Leftarrow$ Let S be an independent set of size k

1. S must contain exactly one vertex from each clause
2. S cannot contain vertices labeled by conflicting literals
3. Thus, it is possible to obtain a truth assignment that makes in the literals in S true; such an assignment satisfies one literal in every clause

